

要件定義書

miraiCP 腕ジェスチャー × 3D オブジェクト操作システム

ver. 1.1 2026 年 5 月作成 (2026 年 5 月 26 日 更新)

大阪成蹊大学 データサイエンス学部 miraiCP プロジェクト

1. プロジェクト概要

1.1 背景と目的

本プロジェクト（miraiCP）は、両腕に装着した micro:bit（3 軸加速度センサ）と、指先・関節に貼付した物理マーカを PC カメラで認識し、自作 3D オブジェクトサイト上のオブジェクトをジェスチャーで直感的に操作するシステムを開発することを目的とする。

従来のマウス・キーボード操作ではなく、身体動作そのものをインタフェースとする体験の設計・実装を通じて、HCI（Human-Computer Interaction）領域の実践的知見を得ることも目的の一つである。

2. システム全体構成

2.1 構成要素一覧

コンポーネント	仕様・型番	数量	備考
micro:bit v2	BBC micro:bit v2	2 台	左腕・右腕それぞれに装着
物理マーカー（実験 A）	100 均 丸型シール	適量	指先に貼付、HSV 色検出で認識
物理マーカー（実験 B）	マスキングテープ（カラー）	適量	関節ごとに帯状に巻く
PC カメラ	Web カメラ（720p 以上推奨）	1 台	Phase 1 で使用。PC 内蔵カメラも可
スマホカメラ	Bluetooth 接続スマートフォン	1 台	Phase 3 オプション（死角補完用）
ホスト PC	Windows / Mac（Chrome 推奨）	1 台	すべての処理を担当

2.2 レイヤー構成

システムは以下の 4 レイヤーで構成される。

- 入力レイヤー：micro:bit（両腕）＋カメラ（PC / スマホ）
- 通信レイヤー：BLE / USB シリアル（micro:bit → PC）、WebRTC（スマホ → PC）
- 処理レイヤー：データ融合・ジェスチャー認識エンジン（ホスト PC 上で動作）
- 出力レイヤー：自作 3D オブジェクトサイト（Three.js / WebXR）

3. マーカー方式比較実験（Phase 0）

本システムの指認識精度はマーカー方式に大きく依存する。そのため、本実装に先立ち、以下2案の比較実験を行い、採用方式を決定する。

3.1 実験 A：丸シール方式（指先貼付）

- 各指の指先に丸型シール（100 均）を 1 枚貼付
- 認識対象：点（重心座標 x, y のみ）
- 想定課題：指を握る・曲げる動作で死角が発生しやすい
- 技術：OpenCV HSV 色検出

3.2 実験 B：マスキングテープ方式（関節巻き）

- 各指の第 1・第 2 関節にカラーマスキングテープを帯状（幅 5～10mm 程度）に巻く
- 認識対象：帯（線分）として扱えるため、関節角度の推定が可能
- 期待効果：どの角度からでも必ず一部が映るため死角が大幅に減少
- 色付きテープ使用により、OpenCV HSV 色検出と容易に組み合わせ可能
- 帯の方向ベクトルから指の曲げ角度を 1 カメラで推定できる可能性がある

3.3 比較評価項目

評価項目	測定方法	目標値
マーカー認識率	同一環境・同一カメラで各方式を撮影し、フレームあたりの検出成功率を計測	90%以上
死角発生率	指を 10°刻みで曲げたときのマーカー消失頻度を記録	実験 A より低いこと
関節角度推定の可否	実験 B のみ：帯の方向ベクトルから曲げ角度を推定できるか検証	誤差 $\pm 10^\circ$ 以内
誤検出率	背景・肌色・衣服との混同発生率	5%以下
装着の手軽さ	実験セットアップ所要時間を計測	5 分以内

3.4 実験の進め方

1. 同一環境・同一カメラで両方式を撮影（照明・背景を統一する）
2. OpenCV（HSV 色検出）で検出数・検出率を計測
3. 指を 0°～90°まで 10°刻みで曲げ、各角度での検出率を記録
4. 実験 B のみ：帯の方向ベクトルから曲げ角度推定の精度を検証
5. 結果をスプレッドシートにまとめ、本実装の採用方式を決定する

4. 機能要件

F1 センサ入力取得

- micro:bit から 3 軸加速度データ (X, Y, Z) をリアルタイム取得
- 左腕・右腕それぞれ独立して取得・識別 (デバイス ID で区別)
- 通信方式: BLE 優先、USB シリアルをフォールバックとして使用
- サンプリングレート: 50Hz 以上 (遅延低減のため)

F2 指・手の認識 (2 層構造)

認識は「MediaPipe Hands (骨格ランドマーク)」と「物理マーカー (補完)」の 2 層構造で実装する。

【第 1 層: MediaPipe Hands】

- 手全体の 21 点ランドマークをリアルタイム抽出 (各点の x, y, z 座標)
- 各指の曲げ角度、手首の向き、手のひら法線ベクトルを計算
- 照明・背景の影響を受けにくく、素の手でも動作する基盤技術として使用
- 推奨ライブラリ: mediapipe (Python) または MediaPipe Tasks for Web (JS)

【第 2 層: 物理マーカー (補完)】

- Phase 0 実験で採用した方式 (丸シール or マスキングテープ) を実装
- 実験 A 採用時: OpenCV HSV 色検出で指先座標 (点) を抽出、MediaPipe が見失った指先を補完
- 実験 B 採用時: 帯の方向ベクトルも取得し、関節角度を MediaPipe の値とフュージョン
- 最終的な認識データ: 両者を加重平均で統合し、単一の骨格データとして出力

F3 手・腕の 3D モデル生成

画像 (MediaPipe Hands の骨格モデル) に示す「21 点 + 骨格線」のシンプルな構造を 3D 空間に生成する。

- 21 点のランドマークを 3D 座標として配置 (Three.js の SphereGeometry で点を表現)
- 各関節を結ぶ骨格線を TubeGeometry または LineSegments で描画
- 指ごとに色分け (親指: オレンジ / 人差し指: ティール / 中指: パープル / 薬指: ブルー / 小指: ピンク)
- micro:bit の加速度センサから腕全体の向きを推定し、骨格モデルをリアルタイム更新
- 左右それぞれの骨格モデルを独立して管理し、3D シーン上に同時描画

F4 データ融合・ジェスチャー認識エンジン

- micro:bit 加速度データ（腕の動き）＋ カメラ骨格データ（手の形）を統合
- 認識したジェスチャーを操作コマンドに変換し、WebSocket または `postMessage` で 3D サイトに送信
- ジェスチャーが未検出の状態（デッドゾーン）を設け、意図しない操作を防ぐ

F5 3D オブジェクト操作

- ジェスチャーコマンドに応じてオブジェクトの拡大・縮小・回転を適用
- 操作はフレームごとに連続的に更新（バッチ処理ではなくストリーミング処理）
- 複数オブジェクトが存在する場合は、グリップジェスチャーで選択対象を切り替える

5. ジェスチャー定義

以下の 5 種類のジェスチャーを定義する。センサ種別は「腕」 = micro:bit、「手」 = カメラ (MediaPipe)、「融合」 = 両方を示す。

No.	ジェスチャー名	動作	センサ種別	検出方法	3D 操作	難易度
G1	ズームイン (拡大)	両手を外に広げる	融合	両手首間の距離増加 (micro:bit) + 手骨格の広がり角度 (カメラ)	オブジェクト拡大	★★☆
G2	ズームアウト (縮小)	両手を内に寄せる	融合	G1 の逆方向の動き。micro:bit で腕間距離の減少を検知、カメラで手骨格の収縮を確認	オブジェクト縮小	★★☆
G3	回転 (X・Y 軸)	片腕を縦・横に円を描くように動かす	腕	右腕 micro:bit の加速度ベクトル変化方向を回転量にマッピング。縦動作→X 軸回転、横動作→Y 軸回転	指定軸まわりの回転	★★★
G4	ロール (Z 軸回転)	片手首をひねる (回内・回外)	手	カメラで手のひら法線ベクトルを算出。時計回りひねり→正方向、反時計回り→負方向の Z 軸回転	Z 軸まわりの回転	★★★
G5	グリップ (モード切替)	手を握る (グー) / 開く (パー)	手	MediaPipe で全指の第 2 関節角度を検出。全指が閉じた状態 = グリップ ON (操作ロック)	操作ロック・解除 / 対象オブジェクト切替	★★☆

5.1 ジェスチャー設計上の注意点

- G5 (グリップ) は全ジェスチャーの「起動スイッチ」として機能させる。グリップ ON の間のみ G1～G4 を有効にすることで、意図しない操作 (デッドゾーン) を防ぐ。
- G3 の回転精度は micro:bit のドリフトに依存するため、定期的なキャリブレーション処理を実装する。
- G1・G2 の拡大縮小量は、腕間距離の変化速度に比例させる (一定速度でなく加速度に応じたスケール変化)。
- 将来の拡張として、平行移動 (両腕を同方向に動かす) や選択 (指差し) の追加を想定しておく。

6. 非機能要件

項目	要件	補足
応答遅延	センサ入力 → 3D 操作反映まで 100ms 以内	体感上の即応性を確保するための上限値
フレームレート	骨格モデル更新 30fps 以上	MediaPipe の処理速度に依存。GPU なしの CPU のみでも達成を目指す
マーカー認識率	通常照明・単色背景で 90% 以上	強い逆光・極端に暗い環境は対象外
物理マーカー補完効果	遮蔽時の補完成功率 80% 以上	MediaPipe が失敗したフレームに対する補完率
動作 OS	Windows 10 以上 / macOS 12 以上	Linux は Chrome 動作確認ができれば対応
対応ブラウザ	Google Chrome（最新版）	WebRTC・WebBluetooth 対応が必須
セキュリティ	カメラ映像はローカル処理のみ。外部サーバーに送信しない	プライバシー配慮

7. 技術スタック

レイヤー	役割	採用技術・ライブラリ	備考
センサ通信	micro:bit → PC	microbit-web-bluetooth / pyserial	BLE 優先、USB フォールバック
カメラ入力	映像取得	WebRTC (PC) / Bluetooth 映像転送 (スマホ)	Phase 3 でスマホ追加
指認識 (第 1 層)	骨格ランドマーク抽出	MediaPipe Hands (Python or JS)	21 点 x,y,z を取得
指認識 (第 2 層)	物理マーカー検出	OpenCV (HSV 色検出)	Phase 0 実験で方式決定後に実装
データ融合	ジェスチャー認識	Python (FastAPI) or Node.js	WebSocket でブラウザに送信
3D オブジェクト	3D サイト	Three.js (+ WebXR 対応予定)	骨格モデルと操作対象オブジェクトを同一シーンに配置
通信プロトコル	処理 → 3D サイト	WebSocket or postMessage	ローカル完結
3D モデル生成	腕・手のスキャン	Meshy.ai / スマホ LiDAR / MediaPipe 骨格から自動生成	簡易骨格モデルは MediaPipe で代替も可

8. フェーズ計画

フェーズ	内容	目標成果物	期間目安
Phase 0 (実験)	マーカー方式比較実験 丸シール vs マスキングテープ	実験レポート・採用方式の決定	1～2 週間
Phase 1 (最小実装)	PC カメラ 1 台 + micro:bit 1 台 (片腕) で最小構成を実装 G5 グリップ 検出 → G1 ズームイン の 1 操作を動作させる	動作するデモ (1 ジェスチャー)	2～3 週間
Phase 2 (機能拡張)	両腕対応・全指認識・全 5 ジェスチャー実装 MediaPipe + 物理マーカー融合 手の骨格 3D モデルのリアルタイム描画	全ジェスチャー動作デモ	3～4 週間
Phase 3 (オプション)	スマホカメラを Bluetooth / WebRTC で接続し、死角を補完 複数視点合成によるオクルージョン解消	スマホ接続対応版	2～3 週間

8.1 Phase 1 の最優先タスク

6. micro:bit 1 台を USB シリアルで PC に接続し、加速度データの取得を確認する
7. PC カメラ映像で MediaPipe Hands を起動し、21 点ランドマークの取得を確認する
8. G5 グリップ (手を握る) の検出ロジックを実装し、コンソールに出力する
9. Three.js で 3D オブジェクトを表示し、WebSocket で G1 ズームインコマンドを受け取り拡大動作させる
10. 上記 4 ステップを通して動作させ、デモ動画を撮影する

9. 未確定事項・今後の検討

項目	現状	検討方針
採用マーカー方式	未定（Phase 0 実験で決定）	実験 A（丸シール） vs 実験 B（マスキングテープ）の比較結果にもとづく
腕 3D モデルの精度	骨格モデル（MediaPipe）で代替する予定だが、実スキャンとの比較が未実施	まず MediaPipe 骨格で進め、必要に応じて Meshy.ai や LiDAR スキャンを検討
ジェスチャーの操作体系（詳細）	5 種類を定義済みだが、感度・閾値・スケーリング係数が未定	Phase 1 の動作確認後にチューニング。ユーザーテストを実施する
スマホ Bluetooth 接続方式	WebRTC を候補としているが、接続安定性は未検証	Phase 2 完了後に実験。不安定な場合は USB テザリングを検討
対応する 3D オブジェクトの種類	現時点で不特定。自作サイト上の任意オブジェクトを想定	Phase 1 のデモ後に操作しやすいオブジェクト形状を定義
将来拡張：平行移動操作	現ジェスチャー定義には含まれていない	両腕を同方向に動かす操作として将来追加を検討

10. 用語集

用語	説明
micro:bit	BBC が開発した教育用マイコンボード。3 軸加速度センサ・BLE・LED を搭載
MediaPipe Hands	Google が開発した手の骨格推定ライブラリ。Web カメラ映像から 21 点のランドマークをリアルタイム抽出する
ランドマーク	MediaPipe が検出する手の関節・指先の 3D 座標点。21 点で構成される
HSV 色検出	Hue（色相）・Saturation（彩度）・Value（明度）の 3 成分で色を指定し、特定色のマーカーを抽出する OpenCV の手法
BLE	Bluetooth Low Energy。消費電力が低く、micro:bit と PC の無線通信に使用
WebRTC	ブラウザ間でリアルタイムにビデオ・音声・データを通信する Web の標準 API。スマホカメラ映像を PC に転送する際に使用
Three.js	ブラウザ上で WebGL を使い 3D グラフィクスを描画する JavaScript ライブラリ
WebSocket	ブラウザとサーバー間でリアルタイムの双方向通信を行うプロトコル。ジェスチャーコマンドの送受信に使用
死角（オクルージョン）	カメラの視野外または指・物体の陰に隠れてマーカーが見えなくなる状態
グリップ	手を握る（グー）ジェスチャー。本システムでは操作の ON/OFF スイッチとして機能する
データ融合	micro:bit のセンサデータとカメラの骨格データを組み合わせ、より精度の高い操作情報を生成する処理

－ 以上 －

11. 実装状況（2026 年 5 月 26 日時点）

以下は現時点で実装・動作確認済みのコンポーネントである。

ファイル名	機能	状態	備考
check_mediapipe.py	MediaPipe Hands による手の骨格認識 ・ 21 点ランドマーク取得 ・ 左右手の識別 ・ 骨格の画面描画	動作確認済み	PC カメラ 1 台で両手同時認識可能を確認
gesture_engine.py	G5 グリップ検出 ・ 手を握る（グー）で GRIP ON	動作確認済み	グリップ状態を画面上に赤/緑で表示

- ・手を開く（パー）で
GRIP OFF
- ・状態変化時のみコンソ
ール出力

次の実装予定

server.py — WebSocket サーバー（ジェスチャーコマンドをブラウザに送信）

index.html / app.js — Three.js による 3D オブジェクト表示・操作

G1 ズームイン — 両手を広げる動作でオブジェクト拡大

フィジカル回転 — 指スライド・フリックで慣性付き回転

ゴミ箱削除 — 選択オブジェクトを払ってゴミ箱へ